
Cours n° 5

GRAPHES

I NOUVEAUX TYPES : DICTIONNAIRES, PILES, FILES**1 NOTION DE FILES**

Imaginer que vous vouliez modéliser une file d'attente :

- Les nouveaux venus sont ajoutés en fin de file
- Le premier à passer est le premier de cette file

Cette structure de données est appelée **file** et est assez élémentaire. Elle est composée de quatre grandes manipulations élémentaires :

- « Créer » : Créer la file, au départ celle-ci est vide
- « Enfiler » : ajouter un élément dans la file (à la fin de celle-ci) ;
- « Défiler » : renvoyer le prochain élément de la file (premier de celle-ci), et le retire de celle-ci
- « Tester si la file est vide » ;

Cela s'appuie sur le principe du premier arrivé, premier sorti (FIFO : First in, First out). C'est un principe utilisé dans :

- les files d'attentes donc
- les serveurs d'impression, qui traitent les requêtes dans l'ordre où elles arrivent
- la gestion des transactions qui doivent attendre pour être traitées

2 NOTIONS DE PILES

Imaginer que vous modéliser la pile d'assiettes de votre cuisine :

- Un assiette à ranger est placé en sommet de la pile
- La première assiette à sortir est celle en sommet de la pile.

Cette structure de données est appelée **pile**, et est assez élémentaire. Elle est composée de quatre manipulations élémentaires importantes :

- « Créer » : créer la pile, au départ celle-ci est vide
- « Empiler » : ajouter un élément dans la pile (à la fin de celle-ci) ;
- « Dépiler » : renvoyer le prochain élément de la pile (dernier de celle-ci), et le retire de celle-ci
- « Tester si la pile est vide »

Cette structure s'appuie sur le principe du dernier arrivé, premier sorti (LIFO : Last in, First out). C'est un principe utilisé dans :

- la gestion des appels récurifs
- L'ensemble de la gestion des CTRL+Z (ou UNDO , « Annuler ») que propose de nombreux logiciels, qui correspond à l'annulation de la dernière commande tapée.
- La gestion des erreurs sous Python : Python renvoie à l'écran l'endroit de l'erreur, mais remonte dans les différentes fonctions dans lesquelles se trouve l'erreur.

3 UTILISATION DE DEQUE

On peut bien entendu simuler le comportement de piles et de files à l'aide de listes (comme il l'est d'ailleurs illustré dans ce cours). Cependant, notamment d'un point de vue temps accès mémoire aux données et écriture des données, il est possible d'améliorer les choses grâce à la bibliothèque `collections` et le module `deque`. Le module `deque` est un conteneur se comportant comme une liste avec des ajouts et retraits rapides à chaque extrémité donc idéal pour manipuler des piles et des files.

Il est important de noter que le module `deque` permet de manipuler des objets dits itérables : listes, chaînes de caractères, dictionnaires, tuple, etc. `deque` se prononce "dèque" et est l'abréviation de l'anglais double-ended queue. Il est possible d'ajouter et retirer des éléments par les deux bouts des deque. Celles-ci gèrent des ajouts et des retraits utilisables par de multiples fils d'exécution (thread-safe) et efficaces du point de vue de la mémoire des deux côtés de la deque, avec approximativement la

même performance en $\mathcal{O}(1)$ dans les deux sens. Bien que les objets `list` gèrent des opérations similaires, ils sont optimisés pour des opérations qui ne changent pas la taille de la liste. Les opérations `pop()` et `insert(0, v)` qui changent la taille et la position de la représentation des données sous-jacentes entraînent des coûts de déplacement de mémoire en $\mathcal{O}(n)$.

- Initialiser une deque : `D=deque(iterable)`. Par exemple, `D=deque([5])` ou `D=deque("pile")` (dans ce cas le premier élément est "p" et le dernier élément est "e");
- Ajouter un élément en fin de deque : `D.append(elem)`. Par exemple, `D.append(4)`;
- Ajouter un élément en début de deque : `D.appendleft(elem)`. Par exemple, `D.appendleft(4)`;
- Ajouter plusieurs éléments en fin de deque : `D.extend(iterable)`.
Par exemple, `D.extend([1,2,3])`;
- Ajouter plusieurs éléments en début de deque : `D.extendleft(iterable)`.
Par exemple, `D.extendleft([1,2,3])`;
- Récupérer un élément en fin de deque : `elem=D.pop()`, on précise que cet élément `elem` est automatiquement supprimé du deque `D`.
- Récupérer un élément en début de deque : `elem=D.popleft()`, on précise que cet élément `elem` est automatiquement supprimé du deque `D`;
- Vider le deque créé : `D.clear()`;

4 DICTIONNAIRES

Un autre type de donnée très utile, natif dans Python, est le dictionnaire. À la différence des listes, qui sont indexées par des nombres, les dictionnaires sont indexés par des clés, qui peuvent être de n'importe quel type immuable, les chaînes de caractères et les nombres peuvent toujours être des clés. Des n -uplets peuvent être utilisés comme clés s'ils ne contiennent que des chaînes, des nombres ou des n -uplets. Si un n -uplet contient un objet muable, de façon directe ou indirecte, il ne peut pas être utilisé comme une clé. On ne peut pas utiliser des listes comme clés, car les listes peuvent être modifiées en place en utilisant des affectations par position ou via des `append()` par exemple.

Le plus simple est de considérer les dictionnaires comme des ensembles de paires (`cle : valeur`), les clés devant être uniques (au sein d'un dictionnaire). Une paire d'accolades crée un dictionnaire vide : `{}`. Placer une liste de paires (`cle:valeur`) séparées par des virgules à l'intérieur des accolades ajoute les valeurs correspondantes au dictionnaire, c'est également de cette façon que les dictionnaires sont affichés.

Les opérations classiques sur un dictionnaire consistent à stocker une valeur pour une clé et à extraire la valeur correspondant à une clé. Il est également possible de supprimer une paire avec `del`. Si vous stockez une valeur pour une clé qui est déjà utilisée, l'ancienne valeur associée à cette clé est perdue. Si vous tentez d'extraire une valeur associée à une clé qui n'existe pas, une exception est levée.

Exécuter `list(d)` sur un dictionnaire `d` renvoie une liste de toutes les clés utilisées dans le dictionnaire, dans l'ordre d'insertion (si vous voulez qu'elles soient ordonnées, utilisez `sorted(d)`). Pour tester si une clé est dans le dictionnaire, utilisez le mot-clé `in`.

Un exemple :

```
1 >>> tel = {'jack': 4098, 'sape': 4139}
2 >>> tel['guido'] = 4127
3 >>> tel
4 {'jack': 4098, 'sape': 4139, 'guido': 4127}
5 >>> tel['jack']
6 4098
7 >>> del tel['sape']
8 >>> tel['irv'] = 4127
9 >>> tel
10 {'jack': 4098, 'guido': 4127, 'irv': 4127}
11 >>> list(tel)
12 ['jack', 'guido', 'irv']
13 >>> sorted(tel)
14 ['guido', 'irv', 'jack']
15 >>> 'guido' in tel
16 True
17 >>> 'jack' not in tel
18 False
```

II VOCABULAIRE DES GRAPHES

1 UN GRAPHE

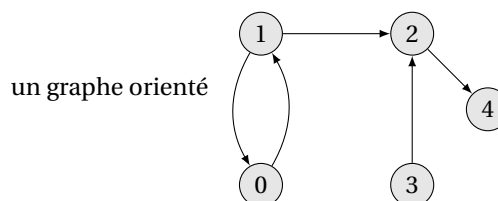
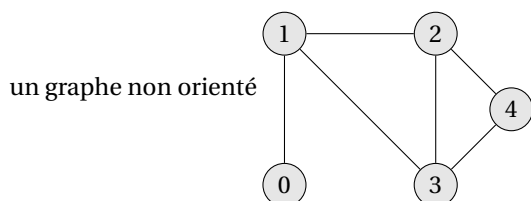
DÉFINITION 1

Un graphe $G = (V, E)$ est couple composé :

- d'un ensemble V de points appelés **sommets** ou vertex en anglais;
- d'un ensemble E de liens appelés **arêtes** ou edge en anglais, l'ensemble E est une relation sur V

On dit que le graphe est **non orienté** si les arêtes non pas de sens. Dans le cas contraire on dit que le graphe est **orienté** et les arêtes sont appelés des **arcs**.

On les représente graphiquement par... des graphes.



Cela permet de modéliser un grand nombre de situations.

EXEMPLES 1.

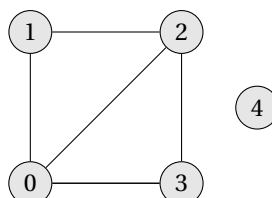
- L'organisation arborescente des dossiers et des fichiers dans un ordinateur est un cas particulier de graphe.
- Les Villes reliées entre elles par des routes forment un graphe, des algorithmes efficaces nous permettent d'être guidés par GPS.
- L'ensemble des liens hypertextes sur le web est un graphe que les moteurs de recherches explorent à l'aide d'algorithmes.
- La résolution d'une grille de Sudoku, ou les labyrinthes, peuvent se modéliser et se résoudre sous forme de recherche dans un graphe.

DÉFINITION 2

- L'ordre d'un graphe est le nombre de sommets. On parle aussi de *noeuds*, *points*, les arêtes sont aussi des *liens*, *lignes*.
- Les sommets reliés par une arête sont les *extrémités* de cette arête. Pour un arc on parle d'extrémité initiale et finale.
- Deux sommets reliés par une arête sont *voisins* ou *adjacents*
- Le nombre de voisin d'un sommet s est son *degré* noté $d(s)$. Un sommet de degré 0 est dit *isolé*.
- Pour un graphe orienté on parle de *degré entrant*, noté $d_-(s)$ et de *degré sortant* noté $d_+(s)$.
- Si tous les sommets sont voisins 2 à 2 on dit que le graphe est *complet*.
- Une arête du type $\{x, x\}$ est une *boucle*.
- Un graphe est *simple* si deux sommets quelconques sont reliés par au plus une arête.

EXEMPLES 2.

- Le graphe non orienté précédent est simple, est de degré 5. le degré du sommet 2 est 3.
- Le graphe orienté précédent est de degré 5, le degré entrant du sommet 2 est 2, et le degré sortant du sommet 2 est 1.
- Pour le graphe suivant le sommet 4 est isolé.

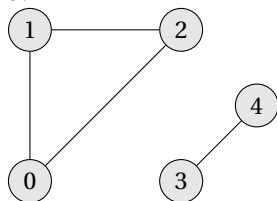


2 CHEMINS

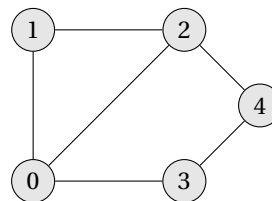
DÉFINITION 3

- Un *chemin* est une séquence $(s_0, s_1, \dots, s_n)$ où deux sommets consécutifs sont adjacents. Ce chemin relie s_0 à s_n . On parle aussi de *chaînes* mais pour des graphes orientés.
- La *longueur* d'un chemin est le nombre d'arêtes utilisées.
- La *distance* entre deux sommets est la longueur minimale d'un chemin reliant ses deux sommets. S'il n'existe pas de chemin reliant les deux sommets, la distance est *infinie*.
- Un *cycle* est un chemin où le sommet de départ et d'arrivée est le même.
- Une graphe est *connexe* quand tout sommet peut être relié à tout autre sommet par un chemin.

EXEMPLES 3.

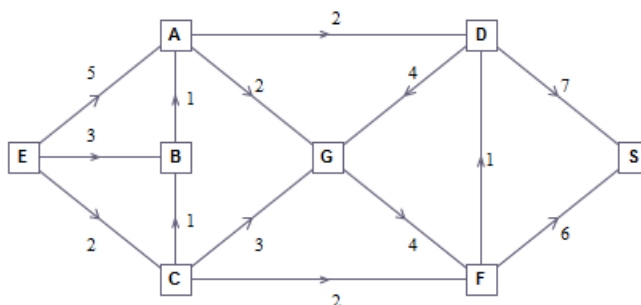


graphe non connexe



la chaîne 1,2,3 est de longueur 2. La distance de 0 à 4 est 2

On peut également ajouter des poids ou des étiquettes aux arêtes d'un graphe. On parle de graphe *pondérée* et de graphe *étiqueté*.



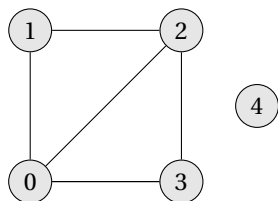
3 REPRÉSENTATION

On utilise essentiellement deux façons de représenter les graphes : la *matrice d'adjacence* et la *liste d'adjacence*.

Pour une *matrice d'adjacence*, on utilise des ... matrices, et donc des tableaux de tableaux en python. Pour une *liste d'adjacence*, on utilise des listes où plutôt des dictionnaires en python.

Plutôt qu'un long discours : des exemples.

EXEMPLE 4.



La matrice d'adjacence est alors :

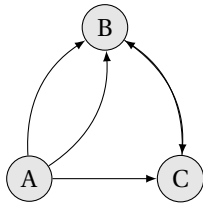
$$A_G = \begin{pmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

Le dictionnaire de la liste d'adjacence est :

graphe = { 0 : [1,2,3], 1 : [0,2], 2 : [0,1,3], 3 : [0,2] }

REMARQUE. La matrice d'adjacence d'un graphe non orienté est symétrique

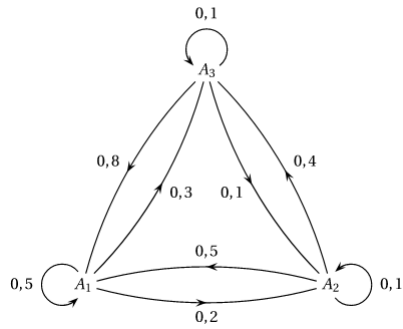
EXEMPLE 5. Dans le cas d'arêtes/arcs multiples, on choisit généralement le nombre d'arêtes.



La matrice d'adjacence est alors :

$$A_G = \begin{pmatrix} 0 & 2 & 1 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}$$

EXEMPLE 6. Dans le cas d'un graphe pondéré on peut adapter en plaçant les poids des arrêtes. C'est extrêmement commun en probabilités, on parle de matrice de transition.



La matrice de transition est alors :

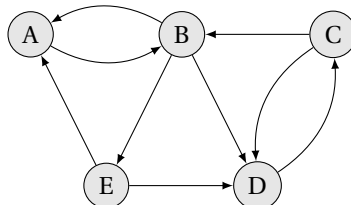
$$T_G = \begin{pmatrix} 0,5 & 0,2 & 0,3 \\ 0,5 & 0,1 & 0,4 \\ 0,8 & 0,1 & 0,1 \end{pmatrix}$$

4 EXERCICES

EXERCICE 1. Vrai ou Faux.

- 1) Un graphe ne peut pas avoir plus de sommets que d'arêtes.
- 2) La matrice d'adjacence d'un graphe à n sommets contient $2n$ éléments.
- 3) Le nombre d'arêtes d'un graphe est la moitié de la somme des degrés de ses sommets.
- 4) Un graphe non orienté d'ordre 5 a au maximum 10 arêtes reliant des sommets distincts.
- 5) Si un sommet est adjacent à tous les autres, le graphe est connexe.
- 6) Un chemin qui passe par tous les sommets et un cycle.

EXERCICE 2. Donner la liste d'adjacence et la matrice d'adjacence du graphe suivant :



EXERCICE 3. Tracer des graphes associés aux matrices d'adjacences données

1) Un graphe non orienté avec $M = \begin{pmatrix} 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \end{pmatrix}$

2) Un graphe orienté avec $M = \begin{pmatrix} 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 \end{pmatrix}$

3) Un graphe orienté avec $M = \begin{pmatrix} 0 & 2 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 2 & 0 & 0 \end{pmatrix}$

EXERCICE 4.

- 1) Écrire une fonction qui prend en argument la matrice d'adjacences d'un graphe (donc une liste de listes en python) et renvoi le nombre d'arêtes de ce graphe.
- 2) Écrire une fonction qui prend en argument la liste d'adjacences d'un graphe (donc un dictionnaire en python) et renvoi le nombre d'arêtes de ce graphe.

III PARCOURS DE GRAPHES

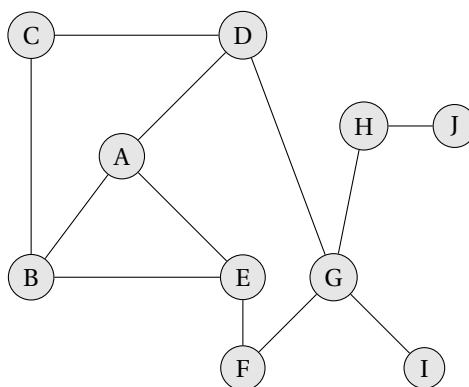
1 QU'EST-CE QUE LE PARCOURS D'UN GRAPHE ?

Le parcours de graphe va permettre d'en déterminer quelques caractéristiques. Celui-ci va donc tout naturellement nous être utile pour vérifier si d'un sommet i , on peut atteindre un autre sommet j . Différentes stratégies sont possibles mais trois sont à connaître :

- Algorithme du parcours en largeur d'un graphe
- Algorithme du parcours en profondeur d'un graphe
- Algorithme de Dijkstra

Dans le cours qui nous intéresse ici, on s'attardera sur les deux premières méthodes, la dernière étant étudiée dans un prochain cours afin de déterminer le chemin (ou chaîne) le (la) plus court(e) dans un graphe pondéré.

Afin d'illustrer les deux algorithmes de parcours de graphe (en largeur et profondeur) et de les expliquer étape par étape, un exemple représentatif des situations rencontrées est proposé :



Graphe exemple

2 QUELS INTÉRÊTS ?

A priori, on peut se poser la question de l'intérêt de tels algorithmes de parcours de graphe. Il faut bien imaginer que les graphes sont représentatifs de divers réseaux et que ces mêmes réseaux peuvent être composés de plusieurs milliers voire millions de sommets. Il n'est pas donc pas forcément toujours possible humainement de déterminer le chemin de connexion entre deux sommets potentiellement distants l'un de l'autre. Il peut paraître également compliqué de déterminer une stratégie de chemin le plus court entre des points distants avec de nombreuses possibilités d'accès (problème du nombre de combinaisons possibles : explosion combinatoire). L'outil informatique est donc à privilégier pour ces cas d'études.

L'étude de parcours de graphe permet par exemple :

- de déterminer si un graphe est connexe (depuis un nœud i quelconque on peut toujours trouver un chemin menant à un nœud j quelconque) ;
- de déterminer l'existence d'un circuit ou d'un cycle ;
- de découvrir un arbre couvrant ;
- de déterminer à l'aide d'algorithmes plus évolués le flot maximal qu'il peut y avoir entre 2 points (flux autoroutier par exemple).

IV PARCOURS EN LARGEUR - BFS

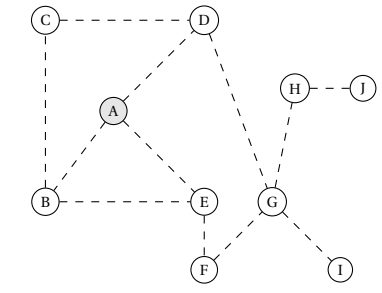
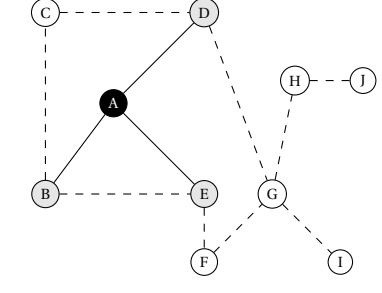
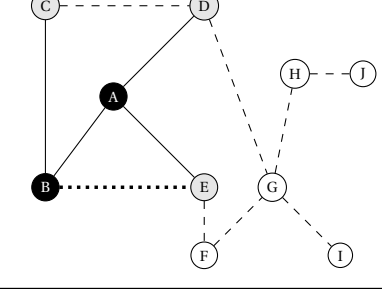
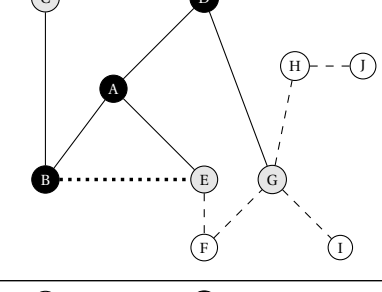
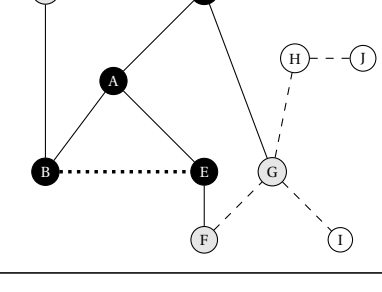
1 PRÉSENTATION DE L'ALGORITHME

Le parcours en largeur d'un graphe consiste par explorer un nœud départ, puis ses successeurs, puis les successeurs non explorés des successeurs, etc. Ainsi, à partir d'un nœud départ *dep*, on liste d'abord les nœuds voisins de *dep* pour ensuite les explorer un par un et ainsi de suite. Mais une illustration sera sans doute plus représentative :

Légende

Nœud non découvert	Nœud exploré	Nœud découvert	Lien exploré	Lien non découvert	Lien découvert mais inutile

Nota : Un nœud exploré est un nœud dont on a découvert tous les voisins (ou successeurs).

Illustration	États d'avancement	Commentaires
	Liste des nœuds explorés $\emptyset$ Liste des nœuds découverts A Liste des nœuds à explorer A	Le nœud A est en cours d'exploration.
	Liste des nœuds explorés A Liste des nœuds découverts A B D E Liste des nœuds à explorer B D E	Le nœud A a été exploré et les nœuds B, D et E ont été découverts. Ils sont ajoutés à la liste des nœuds à explorer.
	Liste des nœuds explorés A B Liste des nœuds découverts A B D E C Liste des nœuds à explorer D E C	Le nœud B est exploré, on découvre alors le nœud C et de nouveau le nœud E. Mais, celui-ci a déjà été découvert, il n'est pas ajouté à la liste des nœuds à explorer.
	Liste des nœuds explorés A B D Liste des nœuds découverts A B D E C G Liste des nœuds à explorer E C G	Le nœud D est exploré, on découvre alors de nouveau les nœuds A et C. Seul G qui le nouveau nœud découvert est ajouté à la liste des nœuds à explorer.
	Liste des nœuds explorés A B D E Liste des nœuds découverts A B D E C G F Liste des nœuds à explorer C G F	Le nœud E est exploré, on découvre alors de nouveau les nœuds A et B. Seul F qui est le nouveau nœud découvert est ajouté à la liste des nœuds à explorer.

	Liste des nœuds explorés A B D E C Liste des nœuds découverts A B D E C G F Liste des nœuds à explorer G F	Le nœud C est exploré. Rien ne se passe car ses nœuds voisins ont déjà été découverts.
	Liste des nœuds explorés A B D E C G Liste des nœuds découverts A B D E C G F H I Liste des nœuds à explorer F H I	Le nœud G est exploré. On découvre deux nouveaux nœuds (H et I) qu'on ajoute à la liste de ceux à explorer.
	Liste des nœuds explorés A B D E C G F Liste des nœuds découverts A B D E C G F H I Liste des nœuds à explorer H I	Le nœud F est exploré. Rien ne se passe.
	Liste des nœuds explorés A B D E C G F H Liste des nœuds découverts A B D E C G F H I J Liste des nœuds à explorer I J	Le nœud H est exploré. On découvre un nouveau nœud : le nœud J.
	Liste des nœuds explorés A B D E C G F H I Liste des nœuds découverts A B D E C G F H I J Liste des nœuds à explorer J	Le nœud I est exploré. Rien ne se passe.
	Liste des nœuds explorés A B D E C G F H I J Liste des nœuds découverts A B D E C G F H I J Liste des nœuds à explorer $\emptyset$	Le nœud J est exploré. L'algorithme s'arrête car la liste des nœuds à explorer est vide.

2 PSEUDO-CODE

L'algorithme du parcours en largeur présenté ci-dessus peut être représenté sous le pseudo-code suivant, en constatant qu'on peut se limiter à deux listes :

```
1 Initialisation;
2 Initialiser la liste file avec le nœud départ ;
3 Initialiser la liste noeuds_decouverts avec le nœud départ ;
4 tant que file n'est pas vide faire
5   | noeud_courant ← Défiler(file) ;
6   | pour chacun des voisins v de noeud_courant faire
7     | si v n'appartient pas à la liste noeuds_decouverts alors
8       | | Ajouter v à la liste noeuds_decouverts;
9       | | Enfiler v à la fin de la liste file;
10    | fin si
11  | fin pour
12 fin
13 return noeuds_decouverts
```

Algorithme 1 : Algorithme de parcours de graphe en largeur : Version 1

3 COMPLEXITÉ

Le graphe est noté $G = (S, A)$.

- Aux lignes 2 et 3 la complexité est constante : $O(1)$;
- Les opérations des lignes 4 et 5 sont réalisés au plus S fois.
- Les opérations et tests des lignes 6 à 9 sont considérés au plus $2A$ fois. En effet, une arête est ce qui relie 2 voisins, ainsi chaque arête est considéré au plus deux fois dans l'algorithme. Par exemple, soient, C et D deux sommets voisins, lorsque C est le nœud courant dans l'algorithme, D est considéré comme un voisin et les diverses opérations (test si D a déjà été visité, Ajouter et Enfiler éventuellement D) sont alors réalisés. Lorsque D est le nœud courant, C est considéré comme un voisin et les divers tests et éventuellement opérations (dans le cas de notre algorithme si C a déjà été visité aucune opération supplémentaire n'est à prévoir) sont alors également effectués.

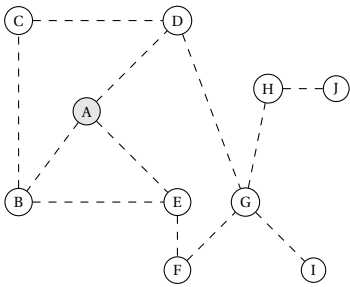
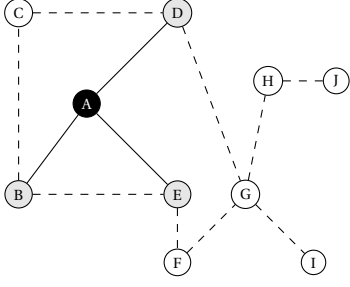
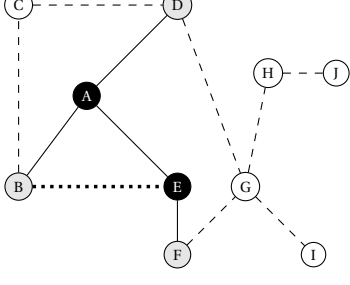
On en déduit donc que la complexité temporelle de l'algorithme est en $O(S + A)$. D'une façon un peu plus triviale, on peut également dire que la complexité est en $O(S \times \max_deg(S))$ avec $\max_deg(S)$ le degré maximal que peut avoir un nœud dans le graphe.

V PARCOURS EN PROFONDEUR - DFS

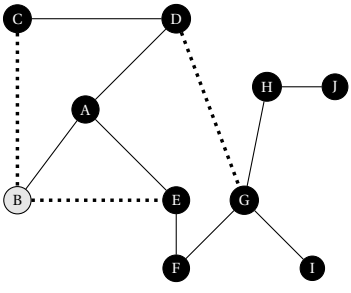
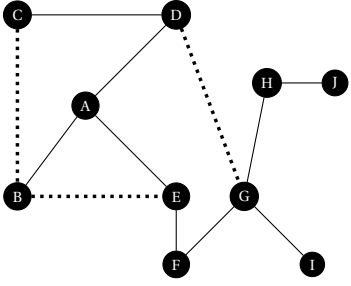
1 PRÉSENTATION DE L'ALGORITHME :

Il existe diverses versions de l'algorithme du parcours en profondeur d'un graphe de façon itérative (et il existe des version récursives) . Ici nous allons voir une version dont le parcours s'appuie sur l'utilisation d'une pile (notion développée un peu plus loin).

On part d'un nœud du graphe, on explore un de ses voisins, puis depuis ce même voisin, on explore un autre voisin qui n'a pas déjà été découvert, etc. Lorsque que l'algorithme amène à ce qu'à partir d'un nœud on ne découvre plus aucun nouveau voisin, on repart du nœud père de celui-ci et on réitère l'algorithme jusqu'à revenir au nœud de départ n'ayant plus de voisins à découvrir.

Illustration	États d'avancement	Commentaires
	Liste des nœuds explorés $\emptyset$ Liste des nœuds découverts A Liste des nœuds à explorer A	Le nœud A est le nœud de départ.
	Liste des nœuds explorés A Liste des nœuds découverts A B D E Liste des nœuds à explorer B D E	Le nœud A est exploré. Les nœuds B, D et E sont découverts. Ils sont ajoutés à la liste des nœuds à explorer.
	Liste des nœuds explorés A E Liste des nœuds découverts A B D E F Liste des nœuds à explorer B D F	Le nœud E est exploré et les nœuds A, B et F ont été découverts. On ajoute uniquement à la liste des nœuds à explorer le nœud E.

	Liste des nœuds explorés A E F Liste des nœuds découverts A B D E F G Liste des nœuds à explorer B D G	Le nœud F est exploré, seul le nœud G est un voisin encore non découvert.
	Liste des nœuds explorés A E F G Liste des nœuds découverts A B D E F G H I Liste des nœuds à explorer B D H I	Le nœud G est exploré, seuls le nœuds H et I sont ajoutés à la liste des nœuds à explorer car le nœud D a déjà été découvert.
	Liste des nœuds explorés A E F G I Liste des nœuds découverts A B D E F G H I Liste des nœuds à explorer B D H	Le nœud I est exploré. Rien ne se passe.
	Liste des nœuds explorés A E F G I H Liste des nœuds découverts A B D E F G H I J Liste des nœuds à explorer B D J	Le nœud H est exploré. On ajoute le nœud J à la liste des nœuds à explorer.
	Liste des nœuds explorés A E F G I H J Liste des nœuds découverts A B D E F G H I J Liste des nœuds à explorer B D	Le nœud J est exploré. Rien ne se passe.
	Liste des nœuds explorés A E F G I H J D Liste des nœuds découverts A B D E F G H I J C Liste des nœuds à explorer B C	Le nœud D est exploré. On découvre le nœud C comme nouveau nœud à explorer.

	Liste des nœuds explorés A E F G I H J D C Liste des nœuds découverts A B D E F G H I J C Liste des nœuds à explorer B	Le nœud C est exploré. Aucun nouveau nœud n'est découvert.
	Liste des nœuds explorés A E F G I H J D C B Liste des nœuds découverts A B D E F G H I J C Liste des nœuds à explorer $\emptyset$	Le nœud B est exploré. Aucun nouveau nœud n'est découvert. L'algorithme s'arrête car la liste des nœuds à explorer est vide.

La principale différence avec d'autres algorithmes (principalement récursifs), est de passer le nœud courant en exploré avant ou après avoir exploré un de ses fils. L'idée d'utilisation d'une liste de nœud à explorer en choisissant comme nœud courant (ou celui d'exploration) le dernier de cette liste et commun aux différentes versions et diffère avec la méthode de parcours en largeur ou on choisit le premier de la liste.

1.A PSEUDO-CODE ITÉRATIF

```

1 Initialisation;
2 Initialiser la liste des nœuds découverts ;
3 Initialiser la pile d'appel avec le nœud de départ;
4 tant que la pile n'est pas vide faire
5   | noeud_courant ← Dépiler(pile);
6   | Ajouter noeud_courant à la liste des nœuds découverts ;
7   | pour chacun des voisins de v de noeud_courant faire
8     | si v n'appartient pas à la liste noeuds_decouverts alors
9       | | Empiler v à la fin de la pile ;
10    | fin si
11  | fin pour
12 fin
13 return la liste des nœuds découverts

```

Algorithme 2 : Algorithme de parcours de graphe en profondeur itératif

2 COMPLEXITÉ

La complexité de l'algorithme de parcours en profondeur est également en $O(S + A)$.

VI PROPRIÉTÉS DE GRAPHES ET PARCOURS

1 QUELQUES RAPPELS

Un graphe non orienté $G = (S, A)$ est dit **connexe** si pour tout couple de sommets (u, v) , il existe un chemin reliant u et v .

Le graphe utilisé pour illustrer les méthodes de parcours de graphe est connexe.

Dans un graphe non orienté, un **cycle** correspond à une chaîne simple dont les extrémités sont confondues et contenant au moins une arête.

Par exemple, sur le graphe précédent, la chaîne A–B–E–A forme un cycle.

Un **arbre** est un graphe connexe sans cycle.

Voici quelques exemples d'arbres.

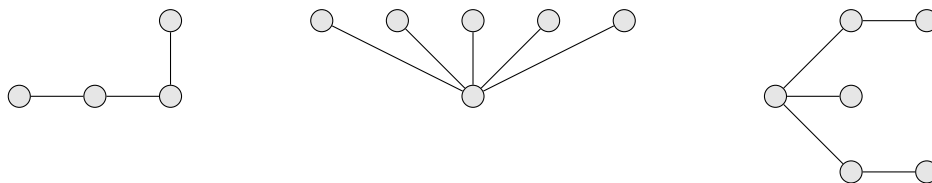


FIGURE 1 – Quelques exemples d'arbres

2 CYCLE DANS UN GRAPHE

Dans un graphe non orienté, la détection de cycles est relativement simple. Il faut tout d'abord effectuer un parcours du graphe et en découvrant au fur et à mesure les sommets, il suffit de vérifier si on découvre un nouveau sommet ou si c'est un sommet déjà visité et qui n'est pas le parent du sommet courant. Dans ce dernier cas, c'est donc qu'il existe un cycle.

Exemple

À partir du graphe utilisé pour illustrer les algorithmes de parcours en largeur et profondeur. Déterminer le nombre de cycles présent dans le graphe proposé.

Dans le graphe précédent, on détecte 3 cycles élémentaires.

- A-B-E-A;
- A-B-C-D-A;
- A-E-F-G-D-A.

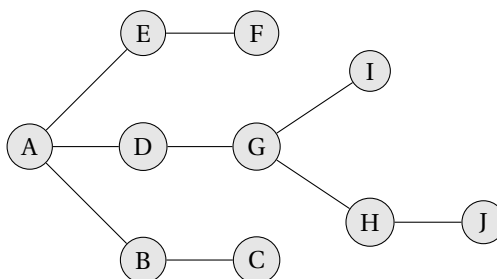
On constate, que ce nombre de cycles correspond aux nombres d'arêtes qui ont été considérés comme « Arête (ou lien) découvert mais inutile » lors des illustrations des parcours de graphe effectués précédemment.

Ainsi, la détection de cycles n'est pas si « dure » que cela et n'a pas de réels impact sur la complexité des algorithmes de parcours.

3 CONNEXITÉ ET ARBRE COUVRANT

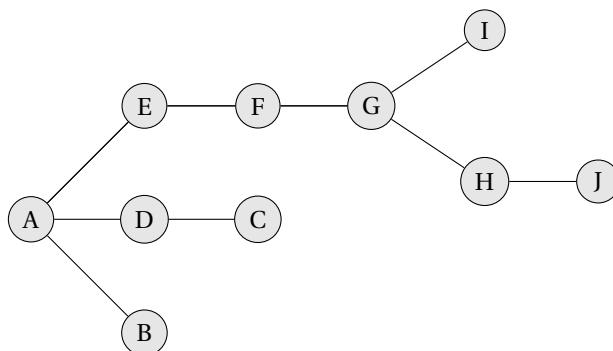
À l'aide des parcours en largeur et en profondeur, on obtient des arbres couvrant le graphe (c'est à dire tout les sommets sont l'arbre). Plus précisément c'est possible si et seulement si le graphe est connexe. Dans le cas contraire on obtient plusieurs arbres (une forêt).

L'arbre couvrant obtenu à partir du parcours en largeur est le suivant :



Arbre obtenu après parcours en largeur

L'arbre couvrant obtenu à partir du parcours en profondeur est le suivant :



Arbre obtenu après parcours en profondeur

Ainsi la détermination d'un arbre couvrant n'est pas unique.

VII DIJKSTRA

On considère des graphes pondérés avec des poids positifs. On cherche à déterminer le plus court chemin entre un sommet initial est un sommet final.

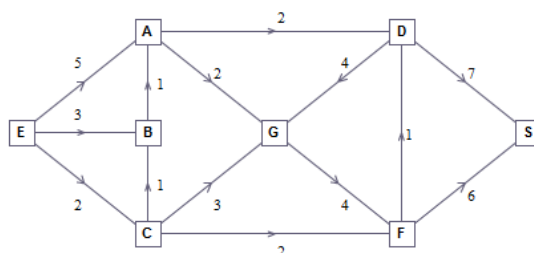
Publié en 1959, il s'agit d'un parcours en largeur légèrement modifié du graphe. A chaque étape on fait le meilleur choix possible : c'est un algorithme glouton.

Le principe est : si le plus court chemin entre deux sommets D et A passe par un sommet I , alors la partie de ce chemin entre D et I est le plus court chemin de D à I , et la partie de ce chemin entre I et A est le plus court chemin entre I et A .

Plutôt qu'une file pour les sommets en attente, on utilise une file de priorité : on extrait le sommet en attente ayant la priorité, c'est à dire celui qui correspond à la distance minimale.

A l'initialisation, on donne à chaque sommet une distance infinie.

Prenons un exemple : On veut déterminer le plus court chemin partant de E pour aller à S .



Un moyen simple d'appliquer l'algorithme est de construire un tableau donnant les distances à chaque sommet découvert en choisissant comme sommet suivant celui dont la distance à E est la plus petite.

Étape	A	B	C	D	F	G	S
0	∞	∞	∞	∞	∞	∞	∞
1	5	3	2	∞	∞	∞	∞
2	5	3		∞	4	5	∞
3	4			∞	4	5	∞
4				6	4	5	∞
5				5		5	10
6						5	10
7							10

Notons qu'on n'obtient pas seulement la distance minimale de E à S , mais la distance minimale de E à chaque point du graphe.

Passons à la mise en machine :

Tout d'abord représenter le graphe

```

1  graphe = {  'E' : [( 'A' ,5), ( 'B' ,3), ( 'C' ,2) ],
2              'A' : [( 'D' ,2), ( 'G' ,2) ],
3              'B' : [( 'A' ,1) ],
4              'C' : [( 'B' ,1), ( 'F' ,2), ( 'G' ,3) ],
5              'D' : [( 'G' ,4), ( 'S' ,7) ],
6              'F' : [( 'D' ,1), ( 'S' ,6) ],
7              'G' : [( 'F' ,4) ],
8              'S' : [] }
```

Ensuite, plutôt que d'utiliser les files de priorité de deque on va simplement utiliser un dictionnaire dont les clés sont les sommets et les valeurs les distances au sommet de départ. On écrit donc une fonction qui permet de déterminer la distance minimale dans les sommets en cours.

Le flottant 'inf' est assez utile : c'est l'infini.

```

1 def minimum(dico):
2     """ dico contient au moins une valeur differente de inf """
3     mini = float('inf')
4     for k in dico :
5         if dico[k] < mini :
6             mini = dico[k]
7             s = k
8     return s

```

Pour la fonction `dijkstra` on utilise deux tableaux : `D` contenant les sommets et les distances minimales au sommet de départ, et `d` les distances pour les sommets "en cours", initialisées à l'infini.

```

1 def dijkstra(G,s):
2     D = {} # tableau final des distances minimales
3     d = { k : float('inf') for k in G } # teableau des distances initiales
4     d[s] = 0 # s sommet de départ
5     while len(d) > 0 : # on termine quand d est vide
6         k = minimum(d) # sommet de distance minimale
7         for j in range(len(G[k])) : # on rergarde les voisins de k
8             v , c = G[k][j] # v un voisin de k, c la distance
9             if not v in D :
10                 d[v] = min(d[v], d[k]+c)
11             D[k] = d[k] # on copie le sommet et la distance minimale dans D
12             del(d[k]) # on supprime le sommet de d
13     return D

```

On teste et :

```

1 In [6]: dijkstra(graphe,'E')
2 Out [6]: {'F': 4, 'A': 4, 'B': 3, 'D': 5, 'E': 0, 'G': 5, 'C': 2, 'S': 10}

```

Le seul problème de l'algorithme de Dijkstra est sa complexité, en $O(n^2)$. Pour un grand nombre de sommets, ça peut être un peu lent.

Une amélioration est l'algorithme A^* : la solution trouvée est une des meilleurs et est obtenue rapidement. Il s'agit d'une méthode heuristique, c'est une méthode qui n'explore pas toutes les possibilités, mais filtre avec des données supplémentaires : mesures, expériences, statistique, hasard.